Project Architecture & System Flow

1. High-Level Overview

This project is a **Real-Time AI Voice Agent** capable of holding natural, low-latency conversations over the phone. It integrates telephony, speech-to-text, large language models, text-to-speech, and a custom knowledge base (RAG) into a seamless pipeline.

2. System Components

A. Telephony & Networking

- **Twilio:** The interface to the traditional telephone network (PSTN). It handles the actual phone call and streams the audio to our server via WebSockets.
- **Ngrok:** A secure tunneling service that exposes our local development server (localhost) to the public internet, allowing Twilio to communicate with it.

B. The Core Server (FastAPI)

- **FastAPI:** A high-performance Python web framework that hosts the WebSocket endpoint (`/voice`).
- **Orchestrator:** The `server.py` script acts as the central brain, managing the state of the conversation, buffering audio, and coordinating between all other AI services.

C. AI Services (The "Organs")

- **The Ears (STT): Groq (Whisper)**. Converts the user's spoken audio into text. We use Groq for its exceptional speed.
- **The Brain (LLM): Groq (Llama 3)**. The intelligence that understands the user's text and generates a response. It is "fed" context from our RAG system.
- **The Mouth (TTS): Piper**. A local, neural text-to-speech engine. It runs on the server itself (no API calls) to generate audio with near-zero latency.
- **The Reflexes (VAD): Silero VAD**. A Voice Activity Detection model that listens to the audio stream 24/7. If it detects the user speaking while the AI is talking, it triggers an immediate "Interruption" (clearing the audio buffer).

D. Knowledge Base (RAG)

- **FAISS:** A vector database that stores mathematical representations (embeddings) of our PDF documents.
- **LangChain:** The framework used to load PDFs, split them into chunks, and retrieve the most relevant chunks based on the user's question.

3. The Data Flow (The Lifecycle of a Conversation)

Step 1: The Call Begins

1. User calls the Twilio phone number.
2. Twilio looks up the "Webhook URL" (your Ngrok URL).
3. Twilio connects to `wss://<ngrok-url>/voice` on our FastAPI server.
4. The connection is established, and the "Introduction" message is generated.

Step 2: User Speaks (Input Processing)

1. **Audio Stream:** Twilio sends raw audio packets (u-law format) to the server via WebSocket.
2. **VAD Check:** The server decodes the audio and feeds it to **Silero VAD**.
    - *If Silence:* The audio is ignored.
    - *If Speech:* The audio is buffered.
3. **Transcription:** Once the user stops speaking (silence detected), the buffered audio is sent to **Groq Whisper API**.
4. **Text Result:** Groq returns the text (e.g., "What are your services?").

Step 3: Thinking (RAG & LLM)

1. **Retrieval:** The server takes the user's text and queries the **FAISS Knowledge Base**.
    - It finds the most relevant paragraphs from the PDFs in the `data/` folder.
    - It boosts scores for documents with matching keywords (e.g., "RFP", "Technical").
2. **Prompting:** The server constructs a prompt for the LLM:
    - *System Instructions:* "You are a helpful assistant..."
    - *Context:* "Here is the information from the PDF: [Content...]"
    - *User Query:* "What are your services?"
3. **Generation:** This prompt is sent to **Groq (Llama 3)**.
4. **Response:** Llama 3 generates a text response (e.g., "We offer Agentic AI automation...").

Step 4: Speaking (Output Processing)

1. **Synthesis:** The text response is sent to the local **Piper** process.
2. **Audio Generation:** Piper generates raw audio bytes (PCM).
3. **Encoding:** The server converts PCM audio back to u-law format (required by Twilio).
4. **Streaming:** The audio is sent back to Twilio via the WebSocket.
5. **Playback:** Twilio plays the audio to the user over the phone.

Step 5: Interruption (Barge-In)

- At any point during Step 4, if **Silero VAD** detects the user speaking:
    1. The server immediately **cancels** the current TTS task.
    2. It sends a "Clear" message to Twilio to stop audio playback instantly.
    3. It starts listening to the user's new input (Step 2).

4. Dependency Graph

Component	Technology	Role	Location
Server	Python / FastAPI	Orchestration	Local / Cloud VM
Tunnel	Ngrok	Public Access	Local
Telephony	Twilio	Call Handling	External API
LLM API	Groq (Llama 3)	Intelligence	External API
STT API	Groq (Whisper)	Transcription	External API
TTS Engine	Piper	Speech Synthesis	**Local Binary**
VAD Engine	Silero	Speech Detection	**Local Library**
Vector DB	FAISS	Knowledge Storage	**Local File**

5. Directory Structure & Key Files

- `server.py` : The main entry point. Handles WebSockets, VAD logic, and ties everything together.
- `rag.py` : Manages the "Brain's Library". Ingests PDFs and finds answers.
- `vad.py` : The "Ears" logic. Wraps Silero VAD for clean speech detection.
- `piper/` : Contains the local TTS engine and voice model.
- `data/` : The folder where you drop PDFs to teach the AI new things.
- `faiss_index/` : The saved "memory" of the PDFs (so we don't have to re-read them every time).